

1. Transformation Thinking in Scala

Modern data engineering pipelines rely on **transformation pipelines**.

Example ETL pipeline:

```
Raw Data
  ↓
Cleaning
  ↓
Filtering
  ↓
Aggregation
  ↓
Business Metrics
```

In Scala this is implemented using collection transformations.

Example:

```
val result =
data
  .filter(...)
  .map(...)
  .groupBy(...)
  .mapValues(...)
```

Spark pipelines follow the same pattern.

Example Spark pipeline:

```
df
  .filter(...)
  .groupBy(...)
  .agg(...)
```

Understanding Scala transformations therefore makes Spark easier to learn.

2. Advanced Collections

Scala provides powerful collection types used to process structured data.

Collection	Use
List	Immutable ordered collection
Set	Unique elements
Map	Key-value storage
Vector	Optimized indexed access
ArrayBuffer	Mutable sequence

Example

```
val departments = List("Engineering","HR","Finance")
```

```
val salaries = List(90000,85000,70000)
```

Transformation example:

```
val updated =  
salaries.map(x => x * 1.1)
```

Result:

```
List(99000,93500,77000)
```

Exercise

Create a list of numbers and:

1 double each number

2 filter numbers greater than 10

Solution

```
val nums = List(5,10,15,20)

val doubled = nums.map(x => x*2)

val filtered = doubled.filter(x => x > 10)

println(filtered)
```

3. Pattern Matching

Pattern matching is a powerful control structure used to match values against patterns.

It is widely used in Scala pipelines for **data classification and transformation logic**.

Example

```
val dept = "Engineering"

dept match {

  case "Engineering" => "Tech"
  case "HR" => "Operations"
  case "Finance" => "Business"
  case _ => "Unknown"

}
```

Example with Data Transformation

```
def deptCategory(dept:String) = {
```

```
dept match {  
  
  case "Engineering" => "Technical"  
  case "HR" => "Operations"  
  case "Finance" => "Business"  
  case _ => "Other"  
  
}  
  
}
```

Lab

Categorize employee departments.

```
val departments =  
List("Engineering", "HR", "Finance", "Legal")  
  
val categories =  
departments.map(dept =>  
  
  dept match {  
    case "Engineering" => "Tech"  
    case "HR" => "Ops"  
    case "Finance" => "Business"  
    case _ => "Other"  
  }  
  
)  
  
println(categories)
```

4. Options (Null Safety)

Scala discourages using null.

Instead it uses the **Option type**.

Option can contain:

Some(value)

None

Example

```
val salary: Option[Int] = Some(90000)
```

Handling safely:

```
salary.getOrElse(0)
```

Why This Matters

Missing values appear frequently in data pipelines.

Using Option prevents runtime errors.

Lab

Handle missing salary values.

```
val salaries = List(Some(90000), None, Some(75000))
```

```
val cleaned =  
salaries.map(_.getOrElse(0))
```

```
println(cleaned)
```

5. Traits

Traits provide reusable functionality similar to interfaces.

They allow multiple inheritance of behavior.

Example

```
trait Logger {  
  
  def log(msg:String) = println(msg)  
  
}
```

Use trait:

```
class Pipeline extends Logger  
  
val job = new Pipeline  
  
job.log("Pipeline started")
```

Exercise

Create a trait called `Validator`.

Method:

```
validateSalary(salary:Int)
```

Return true if `salary > 0`.

6. Error Handling

Scala uses **Try**, **Success**, **Failure** for safe error handling.

This is useful when parsing data.

Example

```
import scala.util.Try

val value = Try("100".toInt)

println(value)
```

Output

```
Success(100)
```

Example with Invalid Data

```
Try("abc".toInt)
```

Output

```
Failure(...)
```

Lab

Safely parse numbers.

```
import scala.util.Try

val data = List("10", "20", "abc", "40")
```

```
val parsed =  
data.map(x => Try(x.toInt).getOrElse(0))  
  
println(parsed)
```

7. Scala Closures

A closure occurs when a function captures variables outside its scope.

Example

```
var factor = 2  
  
val multiply =  
(x:Int) => x * factor
```

The function depends on factor.

Why Closures Matter in Spark

Spark sends closures to worker nodes.

If closures reference large objects or mutable variables, this can cause:

- serialization issues
- incorrect results

Therefore closures should be **simple and deterministic**.

8. Scala → Spark Transformation Mapping

Understanding this mapping is critical.

Scala	Spark
List.map	RDD.map
List.filter	RDD.filter
List.flatMap	RDD.flatMap
List.reduce	RDD.reduce
List.groupBy	groupByKey
List.mapValues	mapValues

Example

Scala transformation:

```
val result =  
numbers.map(x => x*2)
```

Spark equivalent:

```
rdd.map(x => x*2)
```

Example Pipeline

Scala pipeline:

```
employees  
.filter(_.salary > 70000)  
.map(e => (e.dept,e.salary))  
.groupBy(_._1)  
.mapValues(x => x.map(_._2).sum)
```

Spark pipeline:

```
df.filter(col("salary") > 70000)
  .groupBy("dept")
  .agg(sum("salary"))
```

Final Section

End-to-End Scala Transformation Pipeline

Dataset

```
employee_data.csv
1,Alice,Engineering,90000
2,Bob,Engineering,85000
3,Charlie,HR,60000
4,David,Finance,75000
5,Eva,Engineering,95000
6,,Engineering,88000
7,Frank,,65000
8,Grace,Finance,not_available
9,Henry,Engineering,72000
10,Ivy,Engineering,98000
11,Jack,HR,64000
12,Kate,Finance,72000
13,Liam,Engineering,105000
14,Mike,HR,58000
15,Nina,Engineering,87000
```

Dataset issues:

- missing employee names
- missing departments
- invalid salary values
- inconsistent data

Business Requirements

- 1 removes invalid records
- 2 handles missing values
- 3 converts rows into structured objects
- 4 filters employees with salary > 70000
- 5 groups employees by department
- 6 calculates:
 - total salary per department
 - average salary per department
 - number of employees per department

Step 1 – Case Class

```
case class Employee(  
  id:Int,  
  name:String,  
  dept:String,  
  salary:Int  
)
```

Step 2 – Parse Data

```
val raw =  
data.map(_.split(","))
```

Step 3 – Clean Records

Remove records with missing fields.

```
val cleaned =  
raw.filter(x =>  
  x.length == 4 &&  
  x(1).nonEmpty &&  
  x(2).nonEmpty  
)
```

Step 4 – Handle Invalid Salary

```
import scala.util.Try  
  
val parsed =  
cleaned.map(x =>  
  
  Employee(  
    x(0).toInt,  
    x(1),  
    x(2),  
    Try(x(3).toInt).getOrElse(0)  
  )  
  
)
```

Step 5 – Transformation Pipeline

```
val filtered =  
parsed.filter(_.salary > 70000)
```

Step 6 – Aggregation

```
val result =  
filtered  
  .groupBy(_.dept)
```

```

.mapValues(emp =>
  (
    emp.size,
    emp.map(_._salary).sum,
    emp.map(_._salary).sum / emp.size
  )
)

val result = filtered
  .groupBy(_._dept)
  .map { case (dept, employees) =>
    val count = employees.size
    val totalSalary = employees.map(_._salary).sum
    val averageSalary = totalSalary.toDouble / count
    dept -> (count, totalSalary, averageSalary)
  }

println(result)

```

Example Output

```

Engineering -> (5 employees, 452000 total salary, 90400 avg)
Finance -> (2 employees, 147000 total salary, 73500 avg)

```

Dataset for Pipeline

Create file:

```

employee_transactions.csv
emp_id,name,department,city,salary,bonus,experience,join_year
1,Alice,Engineering,New York,90000,10000,5,2019
2,Bob,Engineering,Chicago,85000,8000,4,2020
3,Charlie,HR,New York,60000,5000,6,2018
4,David,Finance,Boston,75000,6000,3,2021
5,Eva,Engineering,Chicago,95000,12000,7,2017
6,,Engineering,Chicago,88000,9000,5,2019
7,Frank,,Boston,65000,4000,4,2020
8,Grace,Finance,Chicago,not_available,5000,6,2018
9,Henry,Engineering,New York,72000,,3,2022
10,Ivy,Engineering,Chicago,98000,11000,8,2016
11,Jack,HR,New York,64000,4000,5,2019
12,Kate,Finance,Boston,72000,5000,6,2018

```

13,Liam,Engineering,Chicago,105000,15000,10,2014
14,Mike,HR,New York,58000,3500,3,2021
15,Nina,Engineering,Chicago,87000,10000,7,2017

Dataset contains:

- missing name
- missing department
- missing bonus
- invalid salary value

Creating Spark Application

Import Spark

```
import org.apache.spark.sql.SparkSession
```

Create Spark Session

```
val spark = SparkSession.builder()  
  .appName("EmployeePipeline")  
  .master("local[*]")  
  .getOrCreate()
```

Import Spark SQL Functions

```
import spark.implicits._  
import org.apache.spark.sql.functions._
```

Loading CSV Data

```
val df =  
  spark.read  
    .option("header", "true")
```

```
.option("inferSchema", "true")  
.csv("employee_transactions.csv")
```

Inspect data.

```
df.show()
```

Lab 1 — Inspect Dataset

```
df.printSchema()  
df.show(5)  
df.count()
```

Applying Schema Explicitly

In production pipelines we avoid `inferSchema`.

Define schema manually.

```
import org.apache.spark.sql.types._  
  
val schema = StructType(List(  
  
  StructField("emp_id", IntegerType, true),  
  StructField("name", StringType, true),  
  StructField("department", StringType, true),  
  StructField("city", StringType, true),  
  StructField("salary", StringType, true),  
  StructField("bonus", IntegerType, true),  
  StructField("experience", IntegerType, true),  
  StructField("join_year", IntegerType, true)  
  
))
```

Load Data with Schema

```
val df =  
spark.read  
  .option("header", "true")  
  .schema(schema)  
  .csv("employee_transactions.csv")
```

Lab 2 — Verify Schema

```
df.printSchema()
```

Data Cleaning Transformations

Step 1 — Handle Invalid Salary

Salary contains "not_available".

Convert salary to integer.

```
val cleanedSalary =  
df.withColumn(  
  "salary_clean",  
  when(col("salary").rlike("[0-9]+"), col("salary").cast("int"))  
  .otherwise(null)  
)
```

Step 2 — Remove Invalid Rows

Remove records with:

- null salary
- null department

- null name

```
val cleaned =  
cleanedSalary  
.filter(col("salary_clean").isNotNull)  
.filter(col("department").isNotNull)  
.filter(col("name").isNotNull)
```

Lab 3 — Data Cleaning

```
cleaned.count()  
cleaned.show()
```

Handling Missing Bonus

Replace null bonus with 0.

```
val bonusFixed =  
cleaned.withColumn(  
  "bonus_clean",  
  coalesce(col("bonus"), lit(0))  
)
```

Derived Columns

Real ETL pipelines often compute derived metrics.

Total Compensation

```
val enriched =  
bonusFixed.withColumn(  
  "total_compensation",
```

```
col("salary_clean") + col("bonus_clean")  
)
```

Experience Category

```
val withLevel =  
enriched.withColumn(  
  "seniority",  
  when(col("experience") >= 8, "Senior")  
    .when(col("experience") >= 5, "Mid")  
    .otherwise("Junior")  
)
```

Lab 4 — Derived Columns

```
withLevel.select(  
  "name",  
  "salary_clean",  
  "bonus_clean",  
  "total_compensation",  
  "seniority"  
)<show()
```

Business Transformation Pipeline

Goal: HR analytics.

1 Department Metrics

```
val deptMetrics =  
withLevel.groupBy("department")
```

```
.agg(  
  
    count("*").alias("employee_count"),  
    sum("salary_clean").alias("total_salary"),  
    avg("salary_clean").alias("avg_salary")  
  
)
```

2 City Salary Analysis

```
val citySalary =  
withLevel.groupBy("city")  
.agg(  
  
    avg("salary_clean").alias("avg_salary")  
  
)
```

3 Top Paid Employees

```
val topEmployees =  
withLevel.orderBy(desc("salary_clean"))  
.limit(5)
```

End-to-End Pipeline

Complete pipeline.

```
val result =  
  
spark.read  
    .option("header", "true")  
    .schema(schema)  
    .csv("employee_transactions.csv")
```

```

.withColumn(
  "salary_clean",
  when(col("salary").rlike("[0-9]+"), col("salary").cast("int"))
)

.filter(col("salary_clean").isNotNull)
.filter(col("name").isNotNull)
.filter(col("department").isNotNull)

.withColumn(
  "bonus_clean",
  coalesce(col("bonus"), lit(0))
)

.withColumn(
  "total_compensation",
  col("salary_clean") + col("bonus_clean")
)

.withColumn(
  "seniority",
  when(col("experience") >= 8, "Senior")
  .when(col("experience") >= 5, "Mid")
  .otherwise("Junior")
)

```

Final Aggregation

```

val analytics =

result.groupBy("department")
.agg(

  count("*").alias("employees"),
  sum("total_compensation").alias("total_comp"),
  avg("salary_clean").alias("avg_salary")
)

```

)

Output

Example:

Engineering -> 7 employees

Finance -> 3 employees

HR -> 3 employees

Final Challenge Exercise

- 1 department with highest average salary
- 2 city with most employees
- 3 employees with salary above department average
- 4 experience distribution by department

Training Lab: End-to-End Spark Data Pipeline (Scala)

Scenario

You work in an **e-commerce analytics team**.

Raw order data arrives daily from multiple systems and contains:

- missing values
- inconsistent formats
- incorrect values
- duplicated records

Your job is to **clean the data and build a transformation pipeline** to produce business metrics.

Step 1 — Dataset (orders.csv)

Create this file:

```
orders.csv
order_id,customer_id,order_date,product,category,price,quantity,city
1001,C101,2024-01-02,Laptop,Electronics,1200,1,New York
1002,C102,2024-01-02,Phone,Electronics,800,2,Chicago
1003,C103,2024-01-03,Tablet,Electronics,500,1,
1004,C101,2024-01-03,Headphones,Electronics,100,3,New York
1005,C104,2024-01-04,Chair,Furniture,200,1,Boston
1006,C105,2024-01-04,Desk,Furniture,350,1,Chicago
1007,C102,2024-01-05,Phone,Electronics,800,1,Chicago
1008,C106,2024-01-05,,Electronics,600,1,San Francisco
1009,C107,2024-01-06,Monitor,Electronics,300,2,
1010,C108,2024-01-06,Desk,Furniture,,1,Boston
1011,C109,2024-01-07,Laptop,Electronics,1200,1,New York
1012,C101,2024-01-07,Mouse,Electronics,50,2,New York
1013,C110,2024-01-08,Chair,Furniture,200,2,Chicago
1014,C111,2024-01-08,Tablet,Electronics,500,1,San Francisco
1015,C112,2024-01-09,Phone,Electronics,800,1,New York
```

This dataset contains issues:

Problem	Example
Missing city	rows 3,9
Missing product	row 8
Missing price	row 10
Duplicate customer orders	many

Step 2 — Pipeline Architecture

Typical Spark Data Pipeline:

```
Raw Data
  ↓
Schema Enforcement
  ↓
Data Cleaning
  ↓
Feature Engineering
  ↓
Business Transformations
  ↓
Aggregation / Metrics
  ↓
Analytics Output
```

Step 3 — Spark Scala Pipeline Code

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.types._
import org.apache.spark.sql.functions._

object EcommercePipeline {

  def main(args: Array[String]): Unit = {

    val spark = SparkSession.builder()
      .appName("Ecommerce Data Pipeline")
      .master("local[*]")
      .getOrCreate()
```

Step 4 — Define Schema

Instead of schema inference, we define schema manually.

```
val orderSchema = StructType(Array(  
  
    StructField("order_id", IntegerType, true),  
    StructField("customer_id", StringType, true),  
    StructField("order_date", StringType, true),  
    StructField("product", StringType, true),  
    StructField("category", StringType, true),  
    StructField("price", DoubleType, true),  
    StructField("quantity", IntegerType, true),  
    StructField("city", StringType, true)  
  
))
```

Why schema matters:

- avoids expensive inference
- ensures type correctness
- improves pipeline reliability

Step 5 — Load CSV

```
val ordersDF = spark.read  
    .option("header", "true")  
    .schema(orderSchema)  
    .csv("data/orders.csv")  
  
ordersDF.show()
```

Step 6 — Data Cleaning

Handle missing values.

```
val cleanedDF = ordersDF
  .filter(col("product").isNotNull)
  .filter(col("price").isNotNull)
  .na.fill("Unknown", Seq("city"))
remove invalid records
fill missing dimensions
```

Step 7 — Feature Engineering

Create new business columns.

```
val transformedDF = cleanedDF
  .withColumn("total_amount", col("price") * col("quantity"))
  .withColumn("order_year", year(to_date(col("order_date"))))
  .withColumn("order_month", month(to_date(col("order_date"))))
```

Now data contains:

column	meaning
total_amount	revenue
order_year	analytics
order_month	dimension
order_month	reporting

Step 8 — Business Filtering

Example business rule:

Ignore small transactions

```
val filteredDF = transformedDF
  .filter(col("total_amount") > 200)
```

Step 9 — Business Aggregations

Revenue per Category

```
val categoryRevenue = filteredDF
    .groupBy("category")
    .agg(sum("total_amount").alias("revenue"))

categoryRevenue.show()
```

Revenue per City

```
val cityRevenue = filteredDF
    .groupBy("city")
    .agg(sum("total_amount").alias("city_revenue"))

cityRevenue.show()
```

Top Customers

```
val topCustomers = filteredDF
    .groupBy("customer_id")
    .agg(sum("total_amount").alias("total_spent"))
    .orderBy(desc("total_spent"))

topCustomers.show()
```

Step 10 — Write Output

```
filteredDF.write
    .mode("overwrite")
    .parquet("output/clean_orders")
```

Why parquet?

- columnar
- compressed
- analytics optimized

Step 11 — Stop Spark

```
spark.stop()  
  
}  
}
```

Lab Tasks

Task 1

Find **total revenue per product**

Task 2

Find **top 3 cities by revenue**

Task 3

Find **average order value**

Task 4

Find **most sold product**

Task 5

Find **monthly revenue trend**

Advanced Challenge (Hard)

city category revenue

Use:

```
groupBy(city, category)
```

Optional Hard Challenge

Create **customer lifetime value**

CLV = total spent per customer

Then filter:

```
customers spending > 1500
```